Chapter 1 Introduction to Databases

Chapter Objectives

In this chapter you will learn:

- The importance of database systems.
- Some common uses of database systems.
- The characteristics of file-based systems.
- The problems with the file-based approach.
- The meaning of the term "database."
- The meaning of the term "database management system" (DBMS).
- The typical functions of a DBMS.
- The major components of the DBMS environment.
- The personnel involved in the DBMS environment.
- The history of the development of DBMSs.
- The advantages and disadvantages of DBMSs.

Although barely five decades old, database research has left a profound imprint on the economy and society, giving rise to an industry sector now valued at between US$35 and US$50 billion annually. The database system — the central subject of this book — is arguably the most consequential development in the history of software engineering. The database has become the foundational infrastructure of the modern information system, fundamentally reshaping the way countless organizations operate. Its importance has only grown in recent years, driven by dramatic advances in hardware capability, storage capacity, and communications technology, including the rise of the Internet, electronic commerce, business intelligence, mobile communications, and grid computing.

Database technology has been one of the most intellectually vibrant areas in all of computing. Since its emergence, it has served as a catalyst for numerous important advances in software engineering. The field is far from exhausted: many open problems remain, and as database applications grow ever more complex, many of the algorithms currently in use — including those for file storage, file access, and query optimization — will need to be fundamentally reconsidered. These foundational algorithms have already contributed significantly to the discipline of software engineering, and new algorithms developed to address tomorrow's challenges will undoubtedly do the same. In this opening chapter, we introduce the database system.

Given its central importance, the study of database systems now features prominently in computing and business curricula worldwide. This book explores a broad range of issues surrounding the implementation and application of database systems, with particular emphasis on how to design a database. A complete design methodology is presented that should equip the reader to tackle databases ranging from the straightforward to the highly complex.

1.1 Introduction

The database has become so embedded in the fabric of daily life that most people interact with one several times a day without ever being aware of it. To frame our discussion, this section examines several concrete applications of database systems. For the purposes of this introduction, we treat a database as a collection of related data, a database management system (DBMS) as the software that manages and controls access to that data, and a database application as any program that interacts with the database at some point during its execution. The more encompassing term database system refers to the collection of application programs that interact with the database together with the DBMS and the database itself. More precise definitions are provided in Section 1.3.

Purchases from the supermarket

When you pay for goods at your local supermarket, a database is almost certainly involved. The checkout assistant scans each item with a bar code reader linked to a database application. That application uses the bar code to retrieve the item's price from a product database, decrements the stock count, and displays the price at the register. If stock falls below a defined reorder threshold, the system may automatically place a replenishment order. A customer calling the store to inquire about availability can receive an immediate answer because a staff member can query the same database through an application program.

Purchases using your credit card

When you pay by credit card, the merchant typically verifies that your available credit is sufficient to cover the purchase. This check — whether performed by phone or automatically through a card reader — draws on a database that records all of your credit card transactions. The application uses your card number to confirm that the current purchase, combined with your spending to date for the month, does not exceed your credit limit. It also checks that the card has not been reported lost or stolen. Once the transaction is approved, its details are added to the database. Separate database applications handle the generation of monthly statements and the crediting of accounts when payments are received.

Booking a vacation with a travel agent

When you inquire about a vacation, your travel agent may consult several databases containing details of available holidays, flights, and fares. When a booking is confirmed, the database system must coordinate all necessary reservations while preventing conflicts. If only one seat remains on a given flight and two agents attempt to reserve it simultaneously, the system must recognize the collision, allow only one booking to proceed, and immediately

notify the other agent that no seats are available. A separate database is typically used for invoicing.

Using the local library

Your local library almost certainly manages its collection through a database system. The database holds details of all books, registered borrowers, loans, and reservations. A computerized catalogue allows readers to search by title, author, or subject. The system manages reservations, notifying borrowers by mail or email when a requested book becomes available, and sends reminders to those who have not returned items by their due date. A bar code reader, similar to the one at the supermarket, tracks books as they are checked out and returned.

Taking out insurance

When you seek any form of insurance — personal, property, or automobile — your agent may consult several databases maintained by different insurance organizations. The personal details you provide, such as your name, address, age, and lifestyle information, are used by the database system to calculate an appropriate premium. An agent can search across multiple providers to find the most competitive offer.

Renting a DVD

A DVD rental company maintains a database recording the titles it stocks, how many copies exist for each title, whether each copy is currently available or on loan, member details, and the current rental status and expected return date of each item. The database may also store richer metadata about each title, such as director and cast information. This data enables the company to monitor stock utilization and forecast future purchasing decisions based on historical rental patterns.

Using the Internet

A large proportion of websites are powered by database applications behind the scenes. An online bookstore such as Amazon.com, for example, allows customers to browse books by category or author and to purchase them. Underpinning the experience is a database holding book titles, ISBNs, authors, prices, stock levels, sales histories, publisher information, reviews, and detailed descriptions. The database supports cross-referencing: a single book may appear under multiple categories such as computing, programming languages, bestsellers, and recommended titles simultaneously. This cross-referencing also enables Amazon to suggest titles that are frequently purchased alongside the one you are

browsing. As with the credit card example, you can supply payment details to complete a purchase online. Amazon further personalizes the experience for returning customers by maintaining a complete record of previous transactions, including purchases, shipping addresses, and payment details, and greeting them by name with recommendations based on prior activity.

Studying at College

If you are enrolled at a college or university, a database system holds records about you: your personal details, declared major and minor fields, current course enrollments, financial aid information, academic history, and examination results. Separate databases likely exist for managing admissions for the following academic year and for administering staff payroll, including personal details and salary information.

These examples represent only a small fraction of the contexts in which database systems are applied. Although we now take such systems for granted, the technology underlying them is remarkably sophisticated, having been developed and refined over several decades. In the next section, we examine the precursor to the database system: the file-based system.

1.2 Traditional File-Based Systems

It is something of a tradition in comprehensive database textbooks to introduce the subject by reviewing its predecessor, the file-based system. We will not depart from that tradition. Although the file-based approach is largely obsolete today, there are compelling reasons to study it:

Understanding the limitations inherent in file-based systems may help prevent us from replicating those shortcomings in database systems — in other words, we stand to learn from earlier experience. Using the word "mistakes" would be uncharitable, since file-based systems served genuinely useful purposes for many years. Nevertheless, that experience taught us that better approaches to managing data are possible.

If you ever need to migrate a file-based system to a database system, understanding how the file system operates will be invaluable, if not indispensable.

1.2.1 File-Based Approach

File-based system: A collection of application programs that perform services for end-users, such as producing reports, where each program defines and manages its own data.

File-based systems were an early effort to computerize the manual filing system that most organizations relied on. In a typical manual system, physical files are organized to hold correspondence, records, or documents related to projects, products, clients, or employees. These files are labeled and stored in cabinets, which may be locked or housed in secure areas for protection. Most of us maintain something similar at home — folders for receipts, warranties, bank statements, and so on. Retrieving information means physically searching through the system, either sequentially or using some form of indexing to locate items more efficiently.

A manual filing system works reasonably well when the volume of records is modest and the primary operations are simple storage and retrieval. It begins to break down, however, when cross-referencing or any kind of computational processing is required. Consider what it would take for a real estate agency to answer questions such as:

- Which three-bedroom properties with at least an acre of land and a garage are currently for sale?
- Which apartments are available for rent within three miles of the city center?
- What is the average rent for a two-bedroom apartment?
- What is the total annual expenditure on staff salaries?
- How does last month's net income compare to this month's projection?
- What is the expected monthly net income for the coming financial year?

The demand for this kind of information has grown steadily. In many industries, detailed monthly, quarterly, and annual reports are a legal requirement. The manual system is clearly unsuited to this kind of processing. File-based systems emerged in response to industry's need for more efficient data access. Rather than establishing a centralized repository for organizational data, however, these systems adopted a decentralized approach in which each department — with the assistance of a Data Processing (DP) team — stored and controlled its own data. The DreamHome example illustrates what this means in practice.

The Sales Department handles the letting and sale of properties. When a client approaches the department to list a property for rent, a form is completed capturing details such as the property's address, number of rooms, and the owner's contact information (see Figure 1.1(a)). Client inquiries are similarly documented on a separate form (Figure 1.1(b)). With DP assistance, the Sales Department builds an information system to handle rental operations, comprising three files holding property, owner, and client details (Figure 1.2).

The Contracts Department manages the lease agreements associated with rental properties. When a client agrees to rent a property, a form capturing client and property details is prepared by sales staff (Figure 1.3) and passed to Contracts, which assigns a lease number and completes the payment and rental period information. Again with DP support, the Contracts Department develops its own information system consisting of three files — for lease, property, and client details — that overlap substantially in content with those maintained by the Sales Department (Figure 1.4).

Figure 1.5 illustrates this arrangement: each department accesses its own files through application programs written specifically for it. Those programs handle data entry, file maintenance, and the generation of a fixed repertoire of reports. Crucially, the physical structure and storage format of the data files are defined directly within the application code.

Similar patterns emerge in other departments. The Payroll Department, for example, stores:

StaffSalary(staffNo, fName, lName, sex, salary, branchNo)

The Human Resources (HR) Department independently maintains:

Staff(staffNo, fName, lName, position, sex, dateOfBirth, salary, branchNo)

The degree of data duplication across these two departments alone is striking, and this pattern is characteristic of file-based systems broadly.

Before discussing the limitations of this approach in detail, it is worth clarifying the terminology. A file is a collection of records containing logically related data. The `PropertyForRent` file in Figure 1.2, for example, holds six records — one per property. Each record consists of one or more fields, where each field represents a specific characteristic of the real-world object being modeled. For `PropertyForRent`, the fields represent characteristics such as address, property type, and number of rooms.

1.2.2 Limitations of the File-Based Approach

This brief description of traditional file-based systems is sufficient to identify their principal limitations, which are summarized in Table 1.1.

Separation and isolation of data

When data is scattered across separate, independently managed files, accessing information that should logically be available together becomes difficult. For example, generating a list of properties that match the preferences of specific clients requires first extracting a subset of clients whose preferred type is "house," then scanning the `PropertyForRent` file for matching properties below the client's maximum rent threshold. Coordinating the processing of two separate files is inherently complex, and the difficulty multiplies when data from three or more files is needed simultaneously.

Duplication of data

The decentralized structure of file-based systems encouraged — indeed, in many cases necessitated — uncontrolled replication of data. The duplication of property and client details between the Sales and Contracts Departments in Figure 1.5 is a clear example. Uncontrolled duplication is costly for several reasons:

Entering the same data more than once wastes both time and money. Storing it multiple times consumes additional space. Often this redundancy could be avoided through data sharing.

More seriously, duplication endangers data integrity — the consistency of stored information. If a staff member moves house and notifies only the HR Department, their pay advice will continue going to the old address because Payroll was never updated. A more

consequential example: if a promotion and accompanying salary increase are communicated to HR but the update never reaches Payroll, the employee will receive the wrong salary. Detecting and correcting such discrepancies takes time and effort. With no automatic synchronization mechanism between departmental files, such inconsistencies are almost inevitable — and even when updates are communicated between departments, transcription errors can introduce further inaccuracies.

Data dependence

Because the physical structure and storage format of files are embedded within the application code, even minor structural changes are costly to implement. Extending the `PropertyForRent` address field from 40 to 41 characters, for example, sounds trivial but requires writing a one-off conversion program that opens the original file, reads each record, converts it to the new format, writes it to a temporary file, deletes the original, and renames the temporary file. Beyond this, every application program that accesses `PropertyForRent` must be identified, modified to reflect the new structure, and retested — even programs that never use the address field itself are affected merely by accessing the file. This characteristic of file-based systems is known as program–data dependence.

Incompatible file formats

Since the file structure is determined by the application programming language used to create it, files generated by programs written in different languages may be structurally incompatible. A COBOL-generated file and a C-generated file may use entirely different internal representations, making joint processing difficult without custom conversion software. For instance, if the Contracts Department (which programs in COBOL) needs to cross-reference property data with the Sales Department's files (written in C), a developer must first write conversion software to bring both datasets into a common format — a time-consuming and expensive undertaking.

Fixed queries and the proliferation of application programs

From the user's perspective, file-based systems were a major step forward from manual methods, and this success generated growing demand for new and modified queries. However, satisfying that demand was entirely dependent on DP developers, who had to write every report and query from scratch. Two outcomes resulted. In some organizations, the range of available reports became effectively frozen — there was no mechanism for answering spontaneous, unplanned (ad hoc) queries. In other organizations, the volume of files and application programs grew without bound, eventually exceeding what the DP Department could manage with available resources. The result was software that was inadequate, poorly documented, and difficult to maintain. Critical capabilities were frequently absent:

■ Security and data integrity controls were not provided.
■ Recovery mechanisms in the event of hardware or software failure were limited or nonexistent.
■ File access was restricted to one user at a time, with no provision for concurrent shared access by multiple staff members.

Neither outcome was acceptable. A fundamentally different solution was needed.

1.3 Database Approach

All of the limitations described above can be traced to two root causes:

1. The definition of data is embedded within the application programs rather than stored separately and independently.
2. Control over access to and manipulation of data extends no further than what individual application programs choose to enforce.

To overcome these problems, a new approach was required. What emerged were the database and the Database Management System (DBMS). This section provides more formal definitions of these concepts and examines the components typically found in a DBMS environment.

1.3.1 The Database

Database: A shared collection of logically related data and its description, designed to meet the information needs of an organization.

The database is a single, potentially large repository of data that can be accessed simultaneously by many departments and users. Rather than a scattered landscape of isolated files with redundant data, all data items are integrated with a minimum of unnecessary duplication. Ownership of the data shifts from individual departments to the organization as a whole — the database is a shared corporate resource.

Crucially, the database holds not only the operational data itself but also a description of that data. This self-describing character is what makes a database different from a simple collection of files. The description of the data is known as the system catalog (also referred to as the data dictionary or metadata — "data about data"). It is this self-describing quality that enables program–data independence.

The separation of data definitions from application programs in the database approach parallels the philosophy of modern software development, where objects expose a public interface independently of their internal implementation. Users interact with an object through its external interface without knowing — or needing to know — how the object is internally structured. This principle is called data abstraction. One of its key benefits is that the internal representation of data can be changed without affecting application programs, provided the external interface remains stable. In a database context, adding a new field to a record or creating a new file leaves existing application programs unaffected. Only when a field that an application actually uses is removed does that application need to be updated.

The phrase "logically related" in the definition also warrants explanation. When analyzing an organization's information needs, we identify entities, attributes, and relationships. An entity is a distinct object — a person, place, thing, concept, or event — that is to be represented in the database. An attribute is a property that characterizes some aspect of the entity. A relationship is an association between entities. Figure 1.6 shows an Entity–Relationship (ER) diagram for part of the DreamHome case study. It identifies six entities (Branch, Staff, PropertyForRent, Client, PrivateOwner, and Lease), seven relationships (Has, Offers, Oversees, Views, Owns, LeasedBy, and Holds), and six primary attributes — one per entity. The database represents these entities, their attributes, and the logical relationships between them. The ER model is explored in depth in Chapters 12 and 13.

1.3.2 The Database Management System (DBMS)

DBMS: A software system that enables users to define, create, maintain, and control access to the database.

The DBMS is the software layer that mediates between users' application programs and the database itself. A typical DBMS provides the following capabilities:

It allows users to define the database, usually through a Data Definition Language (DDL). The DDL enables users to specify data types, data structures, and constraints governing what data may be stored.

It allows users to insert, update, delete, and retrieve data, typically through a Data Manipulation Language (DML). The centralization of all data and its descriptions enables the DML to offer a general-purpose query facility — a query language. This eliminates the two problematic extremes seen in file-based systems: the rigid restriction to a fixed query set and the explosive proliferation of custom application programs. The most widely used query language is the Structured Query Language (SQL, pronounced "S-Q-L" or sometimes "See-Quel"), which is now both the formal and de facto standard for relational DBMSs. Its importance to the field is reflected in the extensive coverage it receives in this book — Chapters 6 through 9 and Appendix I are devoted to a comprehensive study of SQL.

It provides controlled access to the database through several integrated subsystems:

— a security system that prevents unauthorized access;
— an integrity system that maintains the consistency of stored data;
— a concurrency control system that permits safe simultaneous access by multiple users;
— a recovery control system that restores the database to a consistent state following hardware or software failure;
— a user-accessible catalog containing descriptions of the data stored in the database.

1.3.3 (Database) Application Programs

Application Programs: Computer programs that interact with the database by issuing appropriate requests — typically SQL statements — to the DBMS.

Users interact with the database through application programs that create and maintain data and generate information from it. These programs may be conventional batch applications or, more commonly today, interactive online applications. They may be written in third-generation programming languages or in higher-level fourth-generation languages.

Views

With this rich set of capabilities, a DBMS is an extraordinarily powerful tool. However, power alone does not guarantee that users will find the system easy to work with. In fact, because the shared database contains far more data than any individual user needs, users may find themselves confronted with an overwhelming amount of information. For example, the Contracts Department, which previously saw only the rental data it needed (Figure 1.5), now has access to a database that also includes property types, room counts, and owner details. To address this, the DBMS provides a view mechanism that allows each user to interact with a personalized, restricted window onto the database. A view is, in essence, a curated subset of the database tailored to a particular user's needs.

Views offer several benefits beyond reducing cognitive complexity:

Security. Views can be configured to hide data that certain users should not see. A view accessible to branch managers and the payroll office, for instance, might include salary details, while a separate view for general staff would exclude them.

Customization. Views allow the database to present data in the terminology that is most natural to a given user. The Contracts Department, for example, might prefer to see the monthly rent field labeled as "Monthly Rent" rather than the internal field name `rent`.

Stability. A view can present a consistent picture of the database even as the underlying structure evolves — fields added or removed, relationships restructured, tables renamed or split. As long as the changes do not affect the fields the view depends upon, the view itself remains unchanged. This property directly supports the program–data independence described earlier.

The degree to which these capabilities are fully implemented varies from one DBMS product to another. A DBMS designed for a single personal computer may offer limited concurrency support and only rudimentary security, integrity, and recovery features. In contrast, modern large-scale, multi-user DBMS products provide all of the features described above and considerably more. These systems are extraordinarily complex software artifacts — comprising millions of lines of code and accompanied by extensive documentation — precisely because they must address requirements of a very general and demanding nature. Modern production systems are also expected to deliver near-total reliability and continuous 24/7 availability, even in the face of component failures. The DBMS continues to evolve in

response to changing user needs: emerging applications require support for multimedia content including images, video, and audio, and future requirements will inevitably demand further extensions. The evolution of the DBMS is an ongoing process with no foreseeable endpoint.

1.3.4 Components of the DBMS Environment

Hardware

The DBMS and its associated applications require hardware to run. That hardware may range from a single personal computer to a large mainframe or a distributed network of machines. The specific hardware configuration depends on the organization's requirements and on the DBMS being used — some DBMS products are tightly coupled to specific hardware or operating system platforms, while others run across a wide variety of environments. Every DBMS has minimum requirements for main memory and disk space, but meeting only the minimum may not yield acceptable performance. A representative hardware configuration for DreamHome is shown in Figure 1.9. It consists of a network of smaller servers, with a central server in London running the backend of the DBMS — the component that manages and controls access to the database — and client machines at various branch locations running the frontend — the component that interfaces with end-users. This is called a client–server architecture, discussed further in Section 3.1.

Software

The software component encompasses the DBMS product itself, the application programs, and the underlying operating system, including any network software required when the DBMS operates across a network. Application programs are typically written in third-generation programming languages (3GLs) such as C, C++, C#, Java, Visual Basic, COBOL, Fortran, Ada, or Pascal, or in fourth-generation languages (4GLs) such as SQL embedded within a 3GL. Many DBMS products bundle their own fourth-generation development tools, including non-procedural query languages, report generators, forms generators, graphics generators, and application generators. Use of these tools can substantially increase developer productivity and yield programs that are easier to maintain over time.

Data

From the end-user's perspective, data is arguably the most important component in the DBMS environment. Data acts as the bridge between the machine components and the human ones. The database holds both operational data and metadata — the "data about data." The structural definition of the database is called the schema. Let’s assume a schema consists of four tables: `PropertyForRent`, `PrivateOwner`, `Client`, and `Lease`. The `PropertyForRent` table contains eight attributes: `propertyNo`, `street`, `city`, `zipCode`, `type`, `rooms`, `rent`, and `ownerNo`. The `ownerNo` attribute models the ownership relationship between `PropertyForRent` and `PrivateOwner`. The data also includes the system catalog.

Procedures

Procedures are the documented instructions and rules that govern how the database is designed, administered, and used. All users of the system — both those who interact with it day to day and those who manage it — need documented guidance on how to carry out key tasks, including how to:

■ Log on to the DBMS.
■ Use specific DBMS facilities or application programs.
■ Start and stop the DBMS.
■ Create backup copies of the database.
■ Respond to hardware or software failures, including identifying the failed component, arranging for its repair, and recovering the database once the fault is resolved.
■ Modify the structure of a table, reorganize data across multiple storage devices, improve query performance, or archive data to secondary storage.

People

The final component is the people who interact with and manage the system.

1.3.5 Database Design: The Paradigm Shift

Up to this point we have assumed, without explanation, that the data in our database has a defined structure — four tables. But how was that structure determined? The answer is straightforward: through database design. Carrying out that design, however, is far from simple.

Producing a system that genuinely satisfies an organization's information needs requires a fundamentally different orientation from that of the file-based era, where design decisions were driven by the application needs of individual departments. For the database approach to succeed, the organization must learn to think about the data first and the applications second. This reorientation is sometimes described as a paradigm shift.

The stakes are high. A poorly designed database will generate errors that can distort the information used for decision-making, potentially with serious consequences for the organization. A well-designed database, by contrast, provides accurate, timely, and relevant information that supports sound decision-making efficiently.

Regrettably, database design methodologies are not consistently adopted in practice. Many organizations and individual developers rely very little on formal methodology when designing databases, and this is widely recognized as a major contributing factor to database project failures. Without a structured approach, the time and resources required for a project are routinely underestimated, the resulting databases are often inadequate or inefficient, documentation is sparse, and maintenance becomes a persistent burden.

1.4 Roles in the Database Environment

The fifth component of the DBMS environment identified in the previous section is the people. Four distinct categories of people participate in the database environment: data and database administrators, database designers, application developers, and end-users.

1.4.1 Data and Database Administrators

The database and the DBMS are corporate assets that must be managed with the same care as any other organizational resource. The roles of data and database administration are broadly concerned with the management and governance of the DBMS and its data.

The Data Administrator (DA) bears responsibility for managing the organization's data resource. This encompasses database planning, the development and maintenance of standards, policies, and procedures, and conceptual and logical database design. The DA advises and consults with senior management to ensure that the trajectory of database development aligns with and supports broader corporate objectives.

The Database Administrator (DBA) is responsible for the physical realization of the database. This encompasses physical database design and implementation, security and integrity controls, day-to-day maintenance of the operational system, and ensuring that application performance remains satisfactory for end-users. The DBA role is more technically demanding than the DA role, requiring in-depth knowledge of the target DBMS and the overall system environment. In some organizations these two roles are fulfilled by one person; in others, the strategic importance of data as a corporate resource is reflected in the allocation of dedicated teams to each function. Data and database administration are discussed in greater detail in Section 20.15.

1.4.2 Database Designers

In large database projects, it is useful to distinguish between two types of designer: the logical database designer and the physical database designer.

The logical database designer is responsible for identifying the data to be stored — that is, the entities and their attributes — the relationships between those entities, and the constraints that govern what data may be held in the database. This role demands a thorough and complete understanding of the organization's data and of the rules that govern it, sometimes called business rules. Business rules describe the key characteristics of the data from the organization's perspective. In the DreamHome context, examples include:

- A staff member may not manage more than 100 properties for rent or sale simultaneously.
- A staff member may not handle the sale or rental of a property they personally own.
- A solicitor may not act for both the buyer and the seller in the same property transaction.

For the data model to accurately reflect the organization's needs, the logical database designer must engage all prospective users in the design process as early as possible. In this book, the logical design work is divided into two stages:

Conceptual database design, which is conducted independently of any implementation details — the target DBMS, application programming languages, or any other physical considerations.

Logical database design, which targets a specific data model, such as the relational, network, hierarchical, or object-oriented model.

The physical database designer determines how the logical design will be physically implemented. This involves mapping the logical design into a set of tables and integrity constraints, selecting appropriate storage structures and access methods to achieve satisfactory performance, and designing the security measures required to protect the data.

A significant portion of physical database design is specific to the chosen DBMS, and there is often more than one valid way to implement any given mechanism. The physical database designer must therefore be thoroughly familiar with the capabilities and limitations of the target DBMS and be able to evaluate the trade-offs between alternative implementations. Physical design also requires the ability to select storage strategies that account for actual usage patterns. While conceptual and logical design address the question of what data must be represented, physical design addresses the question of how it will be stored and accessed — a distinction that typically calls for different skills, often found in different individuals. Methodologies for conceptual database design are presented in Chapter 16, for logical design in Chapter 17, and for physical design in Chapters 18 and 19.

1.4.3 Application Developers

Once the database has been implemented, the application programs that deliver the required functionality to end-users must be built. This is the responsibility of the application developers. Typically, they work from specifications produced by systems analysts. Each application program contains statements that instruct the DBMS to perform operations on the database — retrieving, inserting, updating, or deleting data. These programs may be written in third-generation or fourth-generation programming languages, as discussed previously.

1.4.4 End-Users

End-users are the clients of the database — the people whose information needs the database was designed, implemented, and maintained to serve. They can be classified according to how they interact with the system:

Naïve users are typically unaware that a DBMS exists. They access the database through specially written application programs designed to make all interactions as simple and

transparent as possible. They issue requests by entering brief commands or making menu selections, with no requirement to understand anything about the underlying database or DBMS. The checkout assistant at a supermarket is a classic example: by scanning a bar code, the assistant triggers an application that looks up the item price, updates the stock count, and displays the result at the register — all invisibly and automatically.

Sophisticated users occupy the opposite end of the spectrum. These individuals are familiar with the structure of the database and with the capabilities the DBMS offers. They may use high-level query languages such as SQL to perform required operations directly, without relying on pre-written application programs. Some sophisticated users may even develop application programs for their own use.

1.5 History of Database Management Systems

As we have seen, the predecessor to the DBMS was the file-based system. The transition was not a discrete event — the file-based approach did not simply stop as the database approach began. In fact, file-based systems survive to this day in certain specialized contexts.

The origins of the DBMS are often traced to the Apollo moon-landing project of the 1960s, initiated in response to President Kennedy's goal of placing a man on the moon by the end of the decade. At that time, no existing system was capable of handling and coordinating the vast quantities of information the project would generate.

In response, North American Aviation (NAA, now Rockwell International), the prime contractor for the project, developed software known as GUAM — Generalized Update Access Method. GUAM was built on the principle that small components combine to form larger ones, which in turn combine to form still larger ones, up to the final assembled product. This structure, resembling an inverted tree, is known as a hierarchical structure. In the mid-1960s, IBM collaborated with NAA to evolve GUAM into what became IMS — the Information Management System. IBM restricted IMS to the management of hierarchical record structures in order to support serial storage devices, most importantly magnetic tape, which was the dominant storage medium of the time. This constraint was later relaxed. Despite being among the earliest commercial DBMSs, IMS remains in active use today as the primary hierarchical DBMS in large mainframe installations.

Also in the mid-1960s, a second significant development emerged: the IDS (Integrated Data Store) system from General Electric, led by the pioneering database researcher Charles Bachmann. IDS gave rise to a new category of database system known as the network DBMS, which exerted considerable influence on the information systems of that era. The network model was developed partly to represent more complex data relationships than hierarchical structures could accommodate, and partly to promote standardization across the industry. To advance that standardization effort, the Conference on Data Systems Languages (CODASYL) — a body representing the U.S. government and the business community — formed a List Processing Task Force in 1965, subsequently renamed the Data Base Task Group (DBTG) in 1967. The DBTG's mandate was to define standard

specifications for an environment that would support database creation and data manipulation. A draft report appeared in 1969, and the first definitive report was published in 1971. The DBTG proposal identified three components:

■ The network schema — the logical organization of the entire database as seen by the DBA — including the database name, the type of each record, and the composition of each record type.
■ The subschema — the portion of the database as seen by a particular user or application program.
■ A data management language for defining data characteristics and structure, and for manipulating data.

For standardization purposes, the DBTG specified three distinct languages: a schema DDL, enabling the DBA to define the schema; a subschema DDL, allowing application programs to specify the portion of the database they require; and a DML for data manipulation.

Although the report was never formally adopted by the American National Standards Institute (ANSI), a number of systems were subsequently built following the DBTG proposal. These are known collectively as CODASYL or DBTG systems. The CODASYL and hierarchical approaches together constitute what are referred to as first-generation DBMSs. These systems suffered from several fundamental weaknesses:

■ Even simple queries required complex navigational programs based on record-by-record processing.
■ Data independence was minimal.
■ There was no widely accepted theoretical foundation.

In 1970, E. F. Codd of the IBM Research Laboratory published his enormously influential paper on the relational data model ("A relational model of data for large shared data banks," 1970). This paper was remarkably well-timed, directly addressing the shortcomings of the earlier approaches. It stimulated the development of numerous experimental relational DBMSs, with the first commercial products appearing in the late 1970s and early 1980s. Of particular significance was the System R project, conducted at IBM's San José Research Laboratory in California during the late 1970s. This project set out to demonstrate the practical viability of the relational model through a working implementation of its data structures and operations. It produced two landmark outcomes:

■ The development of a structured query language — SQL — which became and remains the standard language for relational DBMSs.
■ The production of various commercial relational DBMS products during the 1980s, including DB2 and SQL/DS from IBM, and Oracle from Oracle Corporation.

Today there are several hundred relational DBMS products for both mainframe and personal computer environments, though many push the boundaries of a strict relational definition. Other prominent multi-user relational DBMSs include MySQL from Oracle Corporation, Ingres from Actian Corporation, SQL Server from Microsoft, and Informix from IBM. PC-based examples include Office Access and Visual FoxPro from Microsoft, Interbase from Embarcadero Technologies, and R:Base from R:Base Technologies.

The relational model is not without limitations — most notably its constrained ability to model complex real-world relationships. A substantial body of research has sought to address these limitations. In 1976, Chen introduced the Entity–Relationship model, which has since become the dominant technique for database design and provides the foundation for the design methodology presented in Chapters 16 and 17 of this book. In 1979, Codd himself attempted to remedy shortcomings in his original work with an extended relational model called RM/T, followed in 1990 by RM/V2. The broader effort to develop data models that more faithfully represent the real world has been loosely grouped under the banner of semantic data modeling.

In response to the growing complexity of database applications, two new paradigms emerged: the object-oriented DBMS (OODBMS) and the object-relational DBMS (ORDBMS). Unlike previous models, the precise boundaries of these models are less clearly defined. Together they represent third-generation DBMSs.

The 1990s also witnessed the explosive growth of the Internet, the emergence of the three-tier client-server architecture, and intense pressure to integrate corporate databases with web applications. The late 1990s brought XML (eXtensible Markup Language), a technology with far-reaching consequences for database integration, graphical interfaces, embedded systems, distributed systems, and database applications generally.

Specialized database systems have also proliferated. Data warehouses, for instance, consolidate data drawn from multiple operational sources — possibly maintained by different organizational units — to support comprehensive analytical processing and strategic decision-making based on historical trends. All major DBMS vendors now offer data warehousing solutions. Enterprise Resource Planning (ERP) systems represent another category: application layers built on top of a DBMS that integrate all major business functions of an organization — manufacturing, sales, finance, marketing, shipping, invoicing, and human resources. Prominent ERP systems include SAP R/3 from SAP and PeopleSoft from Oracle.

1.6 Advantages and Disadvantages of DBMSs

The database management system offers a compelling set of potential benefits. It also carries certain costs and risks. This section examines both.

Advantages

The advantages of the database management system approach are:

Control of data redundancy. As discussed the file-based systems waste resources by storing the same information repeatedly across multiple files. The database approach eliminates much of this redundancy by integrating files so that a single authoritative copy of each data item is maintained. It is worth noting, however, that the database approach does not eliminate all redundancy — it controls it. Some key data items must be duplicated to

model relationships, and in certain cases deliberate duplication is justified for performance reasons. The rationale for controlled duplication will become clearer in the chapters that follow.

Data consistency. By eliminating or controlling redundancy, the risk of inconsistencies is substantially reduced. When a data item exists in only one place, any update to its value is made once and immediately reflected for all users. When duplication is present and the system is aware of it, the DBMS can enforce consistency across all copies. It should be noted, however, that many current DBMSs do not automatically guarantee this form of consistency in all situations.

More information from the same amount of data. Integration of previously isolated data sources can unlock additional information that was previously inaccessible. In the file-based system depicted, the Contracts Department has no access to property owner details, and the Sales Department has no visibility into lease information. Once these data sets are integrated in a single database, each department gains access to the other's data, making it possible to derive more meaningful information from the same underlying facts.

Sharing of data. In file-based systems, files are typically owned by the departments that create and use them. A database, by contrast, belongs to the entire organization and can be accessed by all authorized users. New applications can be built on top of existing data, adding only what is not already stored, rather than redefining all data requirements from scratch. New applications can also exploit the DBMS's built-in capabilities — data definition, manipulation, concurrency control, and recovery — rather than implementing these functions themselves.

Improved data integrity. Database integrity refers to the validity and consistency of stored data. Integrity constraints are rules that the data must not violate. Constraints may apply to individual data items within a single record or to relationships between records. An example of an integrity constraint might specify that no staff member's salary may exceed $40,000, or that a branch number recorded in a staff record must correspond to a branch that actually exists. Integration allows the DBA to define such constraints, and the DBMS to enforce them.

Improved security. Database security protects the data from unauthorized access. Without appropriate security controls, integrating data into a single repository would increase vulnerability relative to dispersed file-based systems. However, integration also enables the DBA to define security policies centrally, which the DBMS enforces consistently. Access can be restricted by identity (through usernames and passwords) and by operation type (read, insert, update, or delete). For example, the DBA might have unrestricted access to all data; a branch manager might have access only to data relating to their own branch; and a sales assistant might have access to property data but be blocked from viewing salary information.

Enforcement of standards. Integration allows the DBA to define, and the DBMS to enforce, organization-wide standards. These may include departmental, organizational, national, or international conventions for data formats (to facilitate system interoperability), naming conventions, documentation requirements, update procedures, and access controls.

Economy of scale. Consolidating all operational data into a single database and building a shared set of applications on top of it can reduce costs. Budget that would otherwise be allocated independently to each department for its own file-based system can be pooled, potentially at lower total cost. The combined budget can be used to procure a configuration — whether a single powerful server or a distributed network of smaller machines — that is better matched to the organization's actual needs.

Balance of conflicting requirements. Different users and departments often have competing data needs. Because the database is under the DBA's stewardship, the DBA can make design and operational decisions that optimize the use of resources for the organization as a whole, prioritizing performance for mission-critical applications where necessary, even if that means accepting somewhat lower performance for less critical ones.

Improved data accessibility and responsiveness. Integration means that data spanning departmental boundaries is directly accessible to authorized end-users without intermediary steps. Many DBMSs provide query languages or report generation tools that allow users to pose ad hoc questions and receive answers immediately at their workstation, without requiring a programmer to write a custom application. For example, a branch manager could retrieve all flats with a monthly rent above £400 by submitting the following SQL query:

```
SELECT *
FROM PropertyForRent
WHERE type = 'Flat' AND rent > 400;
```

Increased productivity. The DBMS provides many standard functions — particularly low-level file-handling routines — that would otherwise need to be written into every file-based application individually. This frees programmers to focus on the specific business logic the users require. Many DBMSs also provide fourth-generation development environments with tools that simplify the creation of database applications, further reducing development time and cost.

Improved maintenance through data independence. In file-based systems, the description of data and the logic for accessing it are embedded directly in application code, creating tight coupling between programs and data structures. Any structural change — such as extending an address field from 40 to 41 characters — can require modifications to many programs, even those that do not use the changed field. A DBMS separates data descriptions from application code, making applications immune to changes in data structure.

Increased concurrency. In some file-based systems, concurrent access by multiple users can result in conflicting updates, data corruption, or loss of integrity. A DBMS manages concurrent access and enforces protocols that prevent such interference.

Improved backup and recovery services. File-based systems typically leave backup and recovery responsibilities to individual users, often limited to periodic snapshot backups. If a failure occurs between backups, all work performed in the interim is lost and must be re-entered. Modern DBMSs provide sophisticated facilities that minimize the amount of work lost following a failure.

Disadvantages

The disadvantages of the database approach are:

Complexity. Providing the full range of capabilities expected of a production DBMS makes it an extraordinarily complex piece of software. Database designers, developers, administrators, and end-users must all understand these capabilities to use the system effectively. Inadequate understanding can lead to poor design decisions with serious organizational consequences.

Size. The complexity and breadth of a DBMS translate directly into a large software footprint, requiring significant disk space for installation and substantial main memory to operate efficiently.

Cost of DBMSs. Licensing costs vary widely depending on the environment and the level of functionality required. A single-user desktop DBMS may cost as little as $100, while a large-scale multi-user DBMS serving hundreds of concurrent users may run to $100,000 or even $1,000,000. Annual maintenance contracts — typically calculated as a percentage of the list price — add a recurring cost on top of the initial investment.

Additional hardware costs. The storage demands of the DBMS and the database itself may necessitate the purchase of additional disk capacity. Achieving acceptable performance may require a more powerful machine or even a dedicated server. These hardware requirements entail further capital expenditure.

Cost of conversion. In some cases, the cost of the DBMS and additional hardware is modest compared with the cost of migrating existing applications to run on the new platform. Migration costs include staff retraining and potentially the engagement of specialist consultants to manage the conversion and support ongoing operations. These costs are a primary reason why many organizations remain bound to legacy systems — older, typically inferior systems that they cannot readily abandon.

Performance. A file-based system designed for a specific, narrow application can be highly optimized for that purpose and may therefore perform extremely well. A DBMS, by contrast, is engineered for generality — to serve a wide range of applications simultaneously. This breadth of scope means that some applications may run more slowly under a DBMS than they would under a purpose-built file-based system.

Greater impact of a failure. Centralizing organizational data into a single repository increases the system's vulnerability to disruption. Because all users and all applications depend on the availability of the DBMS, the failure of a critical component can bring the entire organization's operations to a standstill.

Chapter Summary

■ The Database Management System (DBMS) is now the foundational infrastructure of the information system and has fundamentally transformed the operational practices of many organizations. Database systems remain a highly active area of research, with many significant challenges still to be addressed.

■ The predecessor to the DBMS was the file-based system — a collection of application programs that each define and manage their own data to deliver services to end-users, primarily in the form of reports. Although a major improvement over manual filing systems, file-based systems are plagued by significant data redundancy and program–data dependence.

■ The database approach emerged to overcome the limitations of file-based systems. A database is a shared collection of logically related data and a description of that data, designed to meet an organization's information needs. A DBMS is a software system that enables users to define, create, maintain, and control access to the database. An application program is a computer program that interacts with the database through requests — typically SQL statements — issued to the DBMS. The term database system encompasses the application programs, the DBMS, and the database itself.

■ All access to the database passes through the DBMS. The DBMS provides a Data Definition Language (DDL) for defining the structure of the database, and a Data Manipulation Language (DML) for inserting, updating, deleting, and retrieving data.

■ The DBMS provides controlled access to the database through security, integrity, concurrency, and recovery control mechanisms, together with a user-accessible catalog and a view mechanism that presents each user with a tailored subset of the database.

■ The DBMS environment consists of five components: hardware, software (the DBMS, operating system, and application programs), data, procedures, and people. The people include data and database administrators, database designers, application developers, and end-users.

■ The historical roots of the DBMS lie in file-based systems. Hierarchical and CODASYL systems represent first-generation DBMSs — the hierarchical model exemplified by IMS and the network/CODASYL model by IDS, both developed in the mid-1960s. The relational model, proposed by E. F. Codd in 1970, defines second-generation DBMSs and has had a transformative effect on the field. Third-generation DBMSs are represented by Object-Relational and Object-Oriented DBMSs.

■ The advantages of the database approach include control of data redundancy, improved data consistency, data sharing, enhanced security and integrity, and better support for maintenance and concurrency. The disadvantages include increased complexity, higher costs, potential performance trade-offs, and the heightened organizational impact of any system failure.